

Mithesh Project Report

Timeline of work:

All project tasks involved equal contribution from the AI team. Sections marked as 'Primary' represent areas where I was the primary developer and invested the majority of my effort.

Model Testing + Research and Development Phase (Week 1 - Week 3)

Prompt Finalization and BioMistral Execution (Week 4 - Week 9)

Hallucination Detection and Validation (Week 7 - Week 10, Part of Prompt Finetuning)

Formatter Integration and Preprocessing (Team Contribution, Week 14 - Week 15)

Sequential Sentence Mapping and Timestamp Synchronization (Week 16 - Week 18)

Debugging and Integration (Week 3 - Week 21)

Model Testing (Primary)

I started by understanding the BioGPT codebase and testing various language models including Llama, Zephyr, Mistral, and BioMistral. The challenges I faced during this phase were significant. Prompt fine tuning required multiple iterations to get right, token limitations forced me to rethink how much context I could provide, and computing resource constraints meant I had to balance model size against available GPU memory. After extensive testing, BioMistral emerged as the most viable option for our use case.

Research and Development Phase (Team Contribution)

My initial testing approach involved working with video fragments, which I then converted to screenshots. However, I quickly realized that using JSON as the input format for the LLM provided the best tradeoff between token usage and output quality. The structured nature of JSON allowed me to convey precise surgical actions without wasting tokens on redundant visual descriptions. Output quality during this early phase was measured through human review, where I manually examined each generated narration for accuracy and coherence.

Once I settled on JSON as the input format, I began testing across different file sizes to understand how the model handled varying amounts of procedural data. This phase involved

Careful prompt fine tuning where I identified repeat mistakes the model was making and addressed specific edge cases that caused inconsistent outputs. I learned which phrasing patterns triggered hallucinations and which structures kept the model grounded in the actual input data.

Parallel to my work, the team was developing formatter code. Each member contributed different preprocessing approaches, and we tested them all to determine which one handled our surgical data most effectively. The final formatter you see in the notebook represents a combination of everyone's efforts, with the most efficient preprocessing pipeline being selected for integration. This formatter became the foundation for everything that followed.

Formatter Integration and Preprocessing (Team Contribution)

The team's formatter serves two critical functions. First, it encodes anatomical terminology by converting technical object names from the simulation into standardized medical terms. For example, "metatarsal1" becomes "first metatarsal" and "footR" becomes "right foot." This ensures the model receives clinically accurate inputs rather than simulation artifact names. Second, it performs timestamp encoding by sorting all actions chronologically and grouping repetitive actions that occur within a five second window. This reduces redundancy and creates a cleaner narrative structure.

The preprocessing step also standardizes tool names. "BoneSaw" becomes "bone saw" and "Scalpel" becomes "scalpel" to match clinical documentation conventions. The formatter outputs two things: a list of medical steps and a structured prompt ready for BioMistral. The prompt explicitly instructs the model not to invent patient details, anatomical locations, or clinical outcomes beyond what's provided in the input steps.

Prompt Finalization and BioMistral Execution (Team Contribution, Primary)

After the team's preprocessing, I integrated the formatted prompt with BioMistral. The model is configured with specific generation parameters: a maximum of 6000 new tokens, sampling enabled with top-p set to 0.9, and temperature at 0.7. These settings balance creativity with factual grounding. The model generates a clinical narration that transforms the numbered surgical steps into a cohesive operative note paragraph.

This is where the model's biomedical training becomes essential. BioMistral has been pretrained on medical literature, so it naturally uses appropriate clinical phrasing and maintains a professional tone. However, it's not perfect, which is why the next step exists.

Hallucination Detection and Validation (Primary)

I developed a hallucination detection system to catch when the model deviates from the input data. This system uses spaCy for natural language processing and applies two validation checks on every sentence in the generated narration.

First, it checks for action recognition. I maintain a nomenclature dictionary that maps surgical actions like "cutting," "drilling," "sawing," and "incision" to their clinical equivalents. The detector scans each sentence to confirm that a recognized action verb is present. If a sentence describes a procedure without using any known action term, it flags that sentence as potentially hallucinated.

Second, it checks for tool recognition. I have a predefined set of known surgical tools including scalpel, bone saw, drill, forceps, retractor, suction, and irrigator. The detector scans each sentence to verify that an appropriate tool is mentioned. If a sentence claims a procedure was performed but doesn't reference any known tool, or if it references a tool that wasn't in the original input, that's flagged as suspicious.

The output of this detection step is a detailed report. For each sentence, I get the action detected, the tool detected, and any issues found. This allows me to quickly identify which parts of the narration might be unreliable and refine the prompt accordingly. Over multiple iterations, I adjusted the prompt instructions to minimize these flagged sentences, creating a feedback loop that improved output quality.

Enhanced JSON Generation (Team Contribution)

The team developed an enhanced JSON generation system that creates a simplified intermediate representation of the surgical data. Their implementation reads the raw JSON log, sorts all entries by timestamp, and produces a cleaner format where each entry contains the timestamp and a basic narration string structured as "action on target using tool." This work was done in parallel with my hallucination detection efforts, and their code provides the bridge between raw simulation data and the human readable output.

Sequential Sentence Mapping and Timestamp Synchronization (Team Contribution, Primary)

My specific contribution here was building the synchronization mechanism that maps the BioMistral generated narration back to the timestamps. I chose to implement this as a post-processing synchronization step rather than asking the LLM to handle timestamps directly, and this decision was based on empirical observations about LLM limitations.

Even cutting edge language models struggle with large instances of sequential timestamps. During my testing phase, I attempted to include timestamp information directly in the prompts, asking BioMistral to output timestamps alongside each generated sentence. The results were consistently poor. The model would either ignore the timestamp requirements entirely, generate incorrect timestamp values, or break the natural flow of the narration by obsessing over numerical precision at the expense of clinical coherence.

I also experimented with asking the model to produce direct JSON formatted output with embedded timestamps. This approach was even less reliable. The model would frequently produce malformed JSON with missing brackets, inconsistent key names, or timestamp values that didn't correspond to the input data. Even when I provided explicit JSON schema examples in the prompt, the model's attention would split between maintaining valid JSON syntax and generating accurate medical content, resulting in failures in both areas.

The fundamental issue is that LLMs are optimized for generating natural language, not for maintaining strict numerical correspondence across dozens of sequential data points. Asking a model to track 30 or 40 timestamps while simultaneously composing clinically accurate prose introduces cognitive load that degrades both tasks. The model lacks a reliable internal mechanism for preserving exact numerical references over long generation sequences.

By separating timestamp synchronization into its own deterministic post-processing step, I eliminated this source of error entirely. The LLM focuses solely on what it does well: transforming structured surgical steps into coherent clinical narration. Then my synchronization code, which uses simple sequential mapping logic, handles the timestamp alignment with perfect reliability. This separation of concerns proved far more robust than any prompt engineering technique I tested for integrated timestamp generation.

I use NLTK's sentence tokenizer to split the generated paragraph into individual sentences. The mapping logic then pairs each sentence with its corresponding timestamp entry from the enhanced JSON, working on the assumption, and testing, that BioMistral maintains chronological order in its output. I built in validation that compares sentence count to timestamp count and raises warnings when mismatches occur, which helps catch cases where the model skipped steps or invented additional ones.

The synchronized output is a final JSON file where each entry contains a timestamp and its matching sentence from the clinical narration. This format enables the narration to be displayed alongside the simulation video with precise temporal alignment, so each surgical action appears exactly when it occurred in the recorded procedure.

Hardware and Compute Configuration

I ran all experiments using Google Colab Pro with an A100 GPU and high RAM configuration enabled. The A100 provided 80GB of GPU memory, which was essential for loading BioMistral's 7 billion parameter model in float16 precision without running out of VRAM. The high RAM setting gave me 167GB of system memory, which I needed when processing larger JSON files and maintaining multiple model outputs in memory during testing.

The runtime typically consumed around 15GB of GPU memory during inference and 8 to 10GB of system RAM. Usage rate was approximately 7.52 compute units per hour, which translated to reasonable costs for the iterative testing and refinement work I conducted. Colab Pro's 288 compute units allowed me several full days of continuous experimentation before needing to manage usage carefully.

Throughout the project timeline, debugging and integration work ran continuously alongside other development phases. This involved troubleshooting model loading issues, resolving dependency conflicts between different library versions, fixing edge cases in the synchronization logic, and integrating each component as it was completed by different team members.

Recommended next steps for hardware: GCP cloud + New account credits/ Colab Pro

<https://console.cloud.google.com/spanner/instances?project=translatevoc>

https://colab.research.google.com/github/googleapis/langchain-google-spanner-python/blob/main/docs/document_loader.ipynb